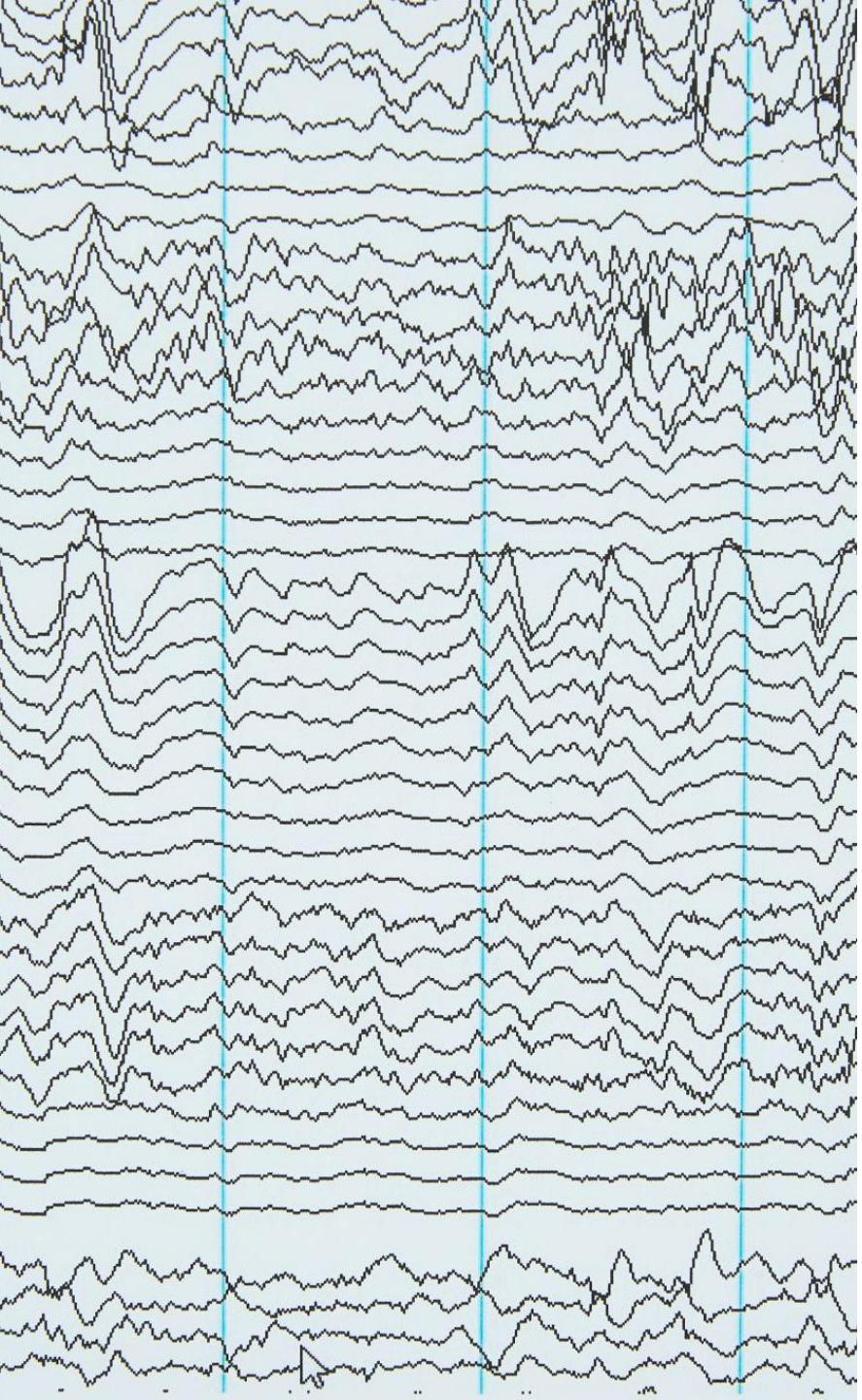
Manual Decomposition & Data Races

Real Work - Vector Addition

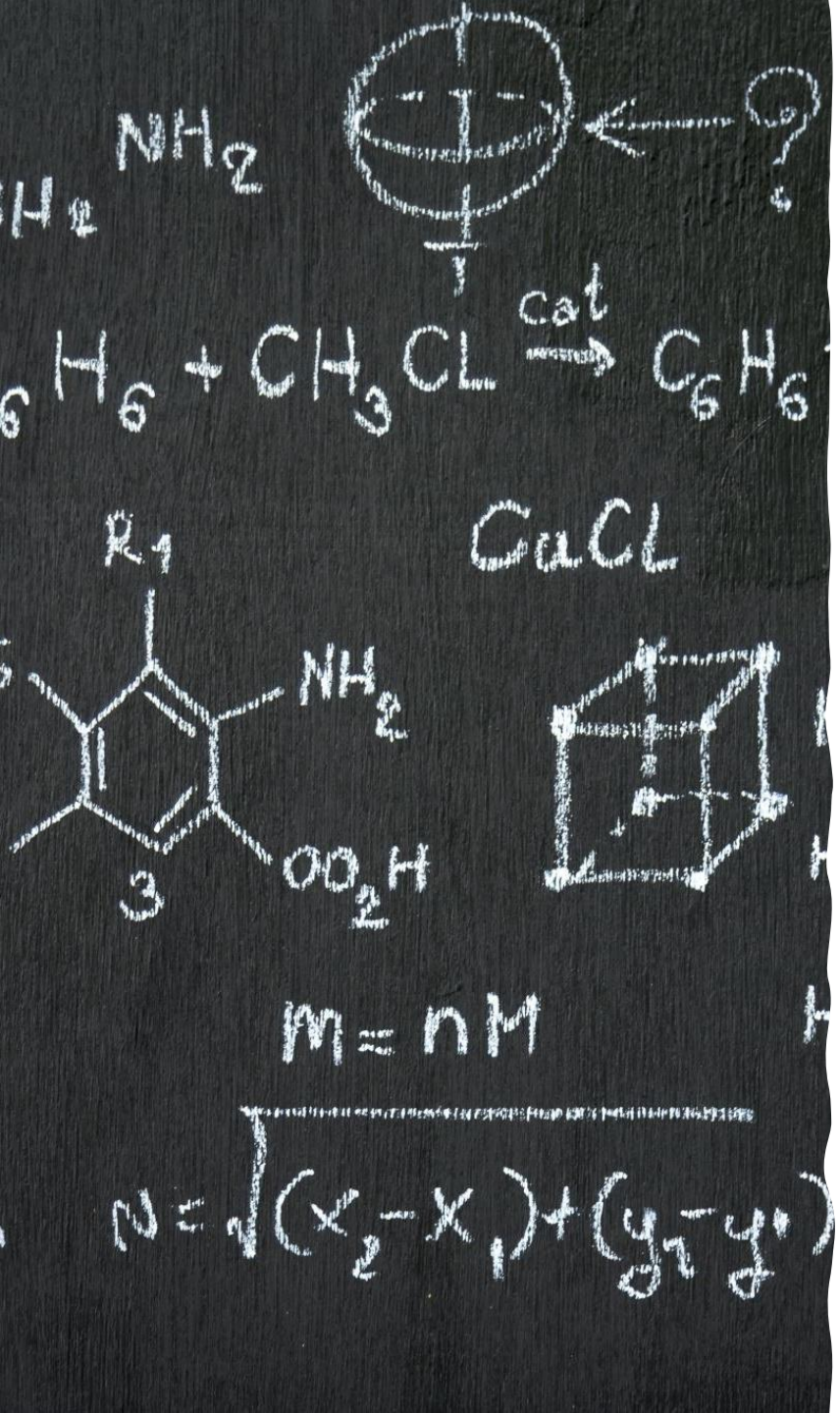
- Let's do math. C = A + B.
- Vectors A and B have 1000 elements.
- **Serial Code:**

```
for (int i = 0; i < 1000; i++) { c[i] = a[i] + b[i]; }
```



How to Parallelize? (SPMD)

- **SPMD:** Single Program, Multiple Data.
  - We run the *same code* on all threads, but they work on *different parts* of the array.
- **Pattern:**
  - Thread 0 does indices 0–249.
  - Thread 1 does indices 250–499.
  - etc.



Manual Decomposition Formula

- If we have N elements and T threads.
  - ChunkSize = N / T
  - MyStart = ID * ChunkSize
  - MyEnd = MyStart + ChunkSize

Manual Parallel Code

```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int nthr = omp_get_num_threads();
    int chunk = 1000 / nthr;
    int start = id * chunk;
    int end = start + chunk;

    for (int i = start; i < end; i++) {
        c[i] = a[i] + b[i];
    }
}
```

- Note: This code assumes 1000 is divisible by T. Real code needs a check for the last chunk.

Example – Manual Loop Calculation

- N=100, Threads=4.
- chunk = 100 / 4 = 25
- **Trace:**
  - **Thread 0:** start = 0*25 = 0. end = 0+25 = 25. (Indices 0–24)
  - **Thread 1:** start = 1*25 = 25. end = 25+25 = 50. (Indices 25–49)
  - **Thread 2:** start = 2*25 = 50. end = 50+25 = 75. (Indices 50–74)
  - **Thread 3:** start = 3*25 = 75. end = 75+25 = 100. (Indices 75–99)
- **Takeaway:** This guarantees no overlap.

Exercise – The "Odd Number" Problem

- **Scenario:** You have N=10 elements and 4 threads.
- **Question:** Using integer division (10 / 4 = 2), what indices does Thread 3 process?
- **Math:**
  - chunk = 2
  - Thread 3: start = 6, end = 8.
- **The Problem:** Indices 8 and 9 are never processed!
- Manual decomposition requires handling the "remainder" (the last few elements) separately.

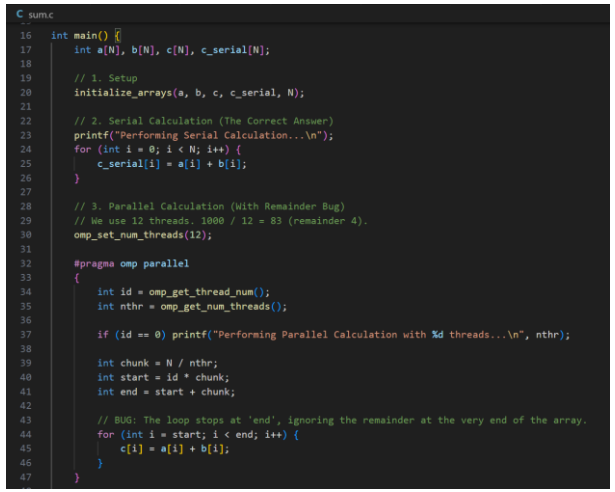
Scheduling problems

sum.c

```c
#include <stdio.h>
#include <omp.h>

#define N 1000

// --- Function to Initialize Arrays ---
void initialize_arrays(int *a, int *b, int *c, int *c_serial, int size) {
    for (int i = 0; i < size; i++) {
        a[i] = i;
        b[i] = 1;
        c[i] = 0;           // Result for Parallel
        c_serial[i] = 0;    // Result for Serial
    }
}
```



```c
int main() {
    int a[N], b[N], c[N], c_serial[N];

    // 1. Setup
    initialize_arrays(a, b, c, c_serial, N);

    // 2. Serial Calculation (The Correct Answer)
    printf("Performing Serial Calculation...\n");
    for (int i = 0; i < N; i++) {
        c_serial[i] = a[i] + b[i];
    }

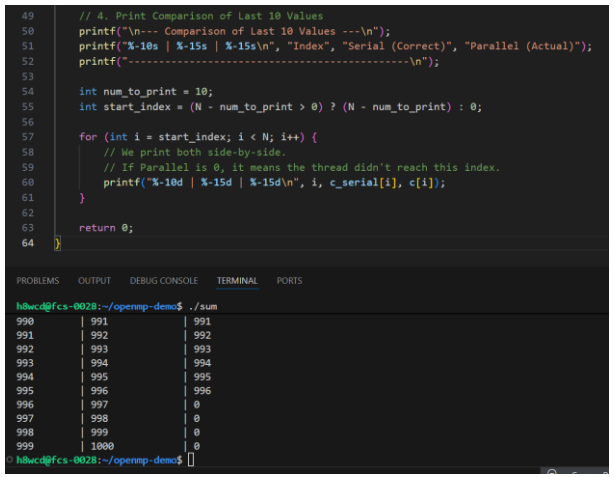
    // 3. Parallel Calculation (With Remainder Bug)
    // We use 12 threads. 1000 / 12 = 83 (remainder 4).
    omp_set_num_threads(12);

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int nthr = omp_get_num_threads();

        if (id == 0) printf("Performing Parallel Calculation with %d threads...\n", nthr);

        int chunk = N / nthr;
        int start = id * chunk;
        int end = start + chunk;

        // BUG: The loop stops at 'end', ignoring the remainder at the very end of the array.
        for (int i = start; i < end; i++) {
            c[i] = a[i] + b[i];
        }
    }
```



```c
    // 4. Print Comparison of Last 10 Values
    printf("\n--- Comparison of Last 10 Values ---\n");
    printf("%-10s | %-15s | %-15s\n", "Index", "Serial (Correct)", "Parallel (Actual)");
    printf("-------------------------------------------------\n");

    int num_to_print = 10;
    int start_index = (N - num_to_print > 0) ? (N - num_to_print) : 0;

    for (int i = start_index; i < N; i++) {
        // We print both side-by-side.
        // If Parallel is 0, it means the thread didn't reach this index.
        printf("%-10d | %-15d | %-15d\n", i, c_serial[i], c[i]);
    }

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ ./sum
990        | 991             | 991
991        | 992             | 992
992        | 993             | 993
993        | 994             | 994
994        | 995             | 995
995        | 996             | 996
996        | 997             | 0
997        | 998             | 0
998        | 999             | 0
999        | 1000            | 0
h8wcd@fcs-0028:~/openmp-demo$
```

sum2.c

```
11  int main() {
28      // 3. Parallel Execution (Manual Block with Fix)
29      omp_set_num_threads(12); // Force remainder (1000 % 12 != 0)
30
31      #pragma omp parallel
32      {
33          int id = omp_get_thread_num();
34          int nthr = omp_get_num_threads();
35
36          int chunk = N / nthr;
37          int start = id * chunk;
38          int end = start + chunk;
39
40          // --- THE FIX ---
41          // If this is the last thread, extend 'end' to N
42          if (id == nthr - 1) {
43              end = N;
44          }
45
46          for (int i = start; i < end; i++) {
47              c_parallel[i] = a[i] + b[i];
48          }
49      }
50
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ ./sum2
--- Strategy 1: Manual Block Fix ---
SUCCESS: Parallel result matches Serial result.
Last Value c[999] = 1000 (Expected 1000)
h8wcd@fcs-0028:~/openmp-demo$
```

Strategy 1: Manual Block Decomposition (The "Last Thread Catch-All")

- **The Concept:** This is the direct fix to the previous code. We calculate standard chunks, but we add a specific check: "If I am the **last thread**, I must ignore the calculated end and extend my range to the very end of the array."
- **Pros:** Simple to implement.
- **Cons:** The last thread does slightly more work than the others (load imbalance).

Strategy 2: Manual Cyclic Decomposition (Round-Robin)

- **The Concept:** Instead of giving each thread a contiguous block (0-100, 101-200), we deal the iterations like a deck of cards.
  - Thread 0 takes indices: 0, 12, 24...
  - Thread 1 takes indices: 1, 13, 25...
- **Pros:** No complex math (chunk, start, end) is needed. It inherently handles the remainder because the loop simply stops when i < N.
- **Cons:** Poor "Spatial Locality" (cache performance) because threads are accessing memory that is spread out.

```
int main() {
    // 3. Parallel Execution (Manual Cyclic)
    omp_set_num_threads(12);

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int nthr = omp_get_num_threads();

        // Start at 'id', and jump by 'nthr' every time
        for (int i = id; i < N; i += nthr) {
            c_parallel[i] = a[i] + b[i];
        }
    }

    // 4. Comparison
    printf("--- Strategy 2: Manual Cyclic ---\n");
    for (int i = 0; i < N; i++) {
        if (c_parallel[i] != c_serial[i]) errors++;
    }
```

Strategy 3: Automatic Work-Sharing (The Professional Way)

- **The Concept:** We stop manually calculating indices entirely.
  - We use #pragma omp for.
  - OpenMP automatically calculates the start/end points and handles any remainders (usually by distributing them evenly among the first few threads).
- **Pros:** Easiest to write, less error-prone, handles remainders automatically.
- **Cons:** None for simple loops like this.

```c
int main() {
    // 2. Serial Execution
    for (int i = 0; i < N; i++) {
        c_serial[i] = a[i] + b[i];
    }

    // 3. Parallel Execution (Automatic)
    omp_set_num_threads(12);

    #pragma omp parallel //we could combine both pragams as: #pragma omp parallel for
    {
        // We only need one directive here
        #pragma omp for
        for (int i = 0; i < N; i++) {
            c_parallel[i] = a[i] + b[i];
        }
    }

    // 4. Comparison
    printf("--- Strategy 3: Automatic OpenMP For ---\n");
    for (int i = 0; i < N; i++) {
        if (c_parallel[i] != c_serial[i]) errors++;
    }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ gcc -fopenmp sum4.c -o sum4
h8wcd@fcs-0028:~/openmp-demo$ ./sum4
--- Strategy 3: Automatic OpenMP For ---
SUCCESS: Parallel result matches Serial result.
Last Value c[999] = 1000
h8wcd@fcs-0028:~/openmp-demo$
```

The Importance of Independence

- In c[i] = a[i] + b[i], calculation of c[10] does not depend on c[9].
  - This is "Embarrassingly Parallel."
- If c[i] = c[i-1] + a[i], we have a **dependency**. You cannot parallelize this easily!

Visualizing the Dependency (The Chain)

- Why Dependencies Break Parallelism?
  - Imagine 4 workers (Threads) trying to compute this array (c[i] = c[i-1] + a[i]).
    - **Thread 1** is assigned indices 0–249.
    - **Thread 2** is assigned indices 250–499.
    - **The Problem:** Thread 2 needs to calculate c[250] = c[249] + a[250].
      - But c[249] is calculated by **Thread 1**.
      - Thread 2 cannot start its very first calculation until Thread 1 finishes its *entire* chunk.
    - **Result:** The threads essentially run one after another (serialized), but with the added overhead of managing threads. It is often *slower* than serial code.

The "Naive" Failure

- The Trap of #pragma omp parallel for
  - If you blindly add #pragma omp parallel for to this loop:
    - **Race Condition:** Thread 2 reads c[249] before Thread 1 has written it. Thread 2 reads garbage data (usually 0).
    - **Domino Effect:** Since c[250] is wrong, c[251] (which depends on 250) is also wrong. The error propagates to the end of the array.
  - **Key Takeaway:** OpenMP assumes iterations are independent. If you violate this, OpenMP will not stop you, but your answers will be wrong.

The Resolution (Prefix Sum / Scan)

- Transforming the Algorithm
  - This specific dependency (sum = sum + val) is a **Prefix Sum/scan**
    - A computation where each output element is the **sum of all previous elements up to that point**.
  - We cannot break the dependency, but we can change the algorithm.
  - **Strategy:**
    - **Local Sums:** Each thread calculates the sum of its own chunk locally.
    - **Offset Calculation:** We calculate the "offset" each thread needs from the previous thread.
    - **Global Update:** Each thread adds the offset to its values.
  - **Modern Solution:** OpenMP 5.0 introduced the scan directive to handle this automatically!

The Failure (Demonstrating the Bug)

dependency1.c

```c
/* dependency_fail.c */
#include <stdio.h>
#include <omp.h>

#define N 20

int main() {
    int a[N], c_serial[N], c_parallel[N];

    // 1. Initialize 'a' with simple 1s
    for (int i = 0; i < N; i++) {
        a[i] = 1;
        c_serial[i] = 0;
        c_parallel[i] = 0;
    }

    // 2. Serial Calculation (Correct Logic: Prefix Sum)
    // c[0] = 1, c[1] = 2, c[2] = 3...
    c_serial[0] = a[0];
    for (int i = 1; i < N; i++) {
        c_serial[i] = c_serial[i-1] + a[i];
    }
```

```c
int main() {
    // 3. Naive Parallel Attempt (BROKEN)
    // We force 2 threads to ensure a split happens at index 10
    omp_set_num_threads(2);

    // c_parallel[0] needs to be initialized manually for the logic to "run"
    c_parallel[0] = a[0];

    #pragma omp parallel for
    for (int i = 1; i < N; i++) {
        // Dependency: Read c[i-1] then Write c[i]
        c_parallel[i] = c_parallel[i-1] + a[i];
    }

    // 4. Print Comparison
    printf("%-5s | %-10s | %-10s\n", "Idx", "Serial", "Parallel");
    printf("-----------------------------\n");
    for (int i = 0; i < N; i++) {
        char *status = (c_serial[i] == c_parallel[i]) ? " " : "<- ERROR";
        printf("%-5d | %-10d | %-10d %s\n", i, c_serial[i], c_parallel[i], status);
    }

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ ./dependency1
4     | 5          | 5
5     | 6          | 6
6     | 7          | 7
7     | 8          | 8
8     | 9          | 9
9     | 10         | 10
10    | 11         | 11
11    | 12         | 1          <- ERROR
12    | 13         | 2          <- ERROR
13    | 14         | 3          <- ERROR
14    | 15         | 4          <- ERROR
15    | 16         | 5          <- ERROR
```

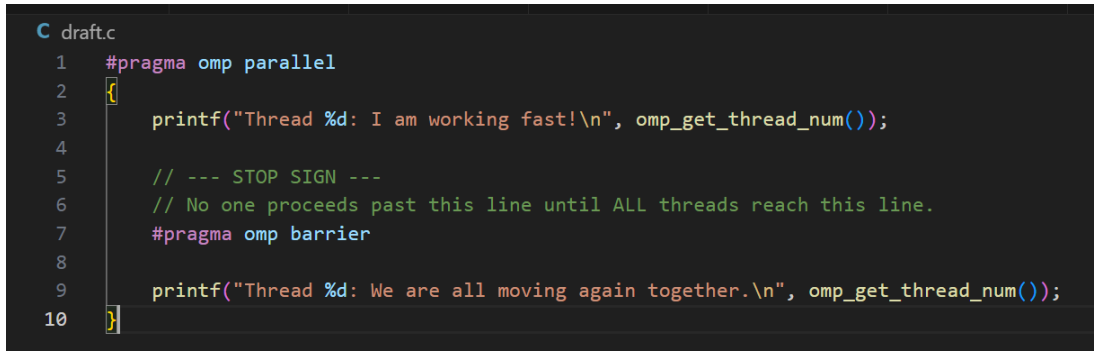
The OpenMP 4.5 Solution (Manual Prefix Sum)

- This method breaks the dependency into three distinct phases:
  - **Phase 1 (Parallel):** Each thread calculates the prefix sum for its own *local* chunk. It stores the total sum of its chunk in a shared offsets array.
  - **Phase 2 (Serial):** One thread (usually the) calculates the prefix sum of those chunk totals to find the global starting point for each thread.
  - **Phase 3 (Parallel):** Each thread adds its global starting offset to every element in its chunk.

This needs to introduce new concepts

The "Wait for Everyone" Checkpoint

- **Concept:**
  - In a parallel region, threads run at different speeds. Some finish their work early, others are slow.
  - **The Danger:** If Thread 1 finishes early and tries to *read* result data that Thread 2 hasn't *written* yet, you get garbage data (Race Condition).
  - **The Solution (barrier):** A stop sign. Every thread that reaches the barrier **must wait** until *every other thread* in the team has arrived.
  - **Analogy:** A hiking group. Fast hikers wait at the trail marker until the slowest person catches up before everyone starts the next section together.



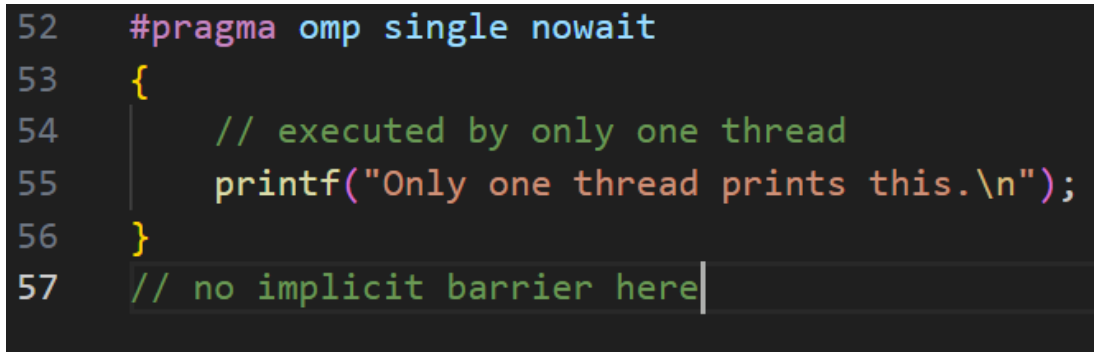
One Worker, Many Watchers

- **Concept:**
  - Sometimes, inside a parallel region, you have a task that should be done exactly **once**.
    - *Examples:* Printing a status update, resetting a counter, or reading a file input.
  - **How it works:**
    - The **first thread** to reach the #pragma omp single **block executes it.**
    - All other threads **wait** at the end of the block (unless you specify nowait).
    - Once the single thread finishes the task, everyone proceeds.
  - **Why not just use if (id == 0)?**
    - single allows *any* available thread to do the work (potentially faster).
    - single has an automatic barrier at the end to ensure safety.

```
13  #pragma omp parallel
14  {
15      // Everyone does this
16      do_heavy_calculation();
17
18      // Only ONE thread does this (e.g., printing progress)
19      #pragma omp single
20      {
21          printf("Phase 1 Complete. Starting Phase 2...\n");
22      } // <-- Implicit Barrier here: Everyone waits for the print to finish.
23
24      // Everyone continues
25      do_next_calculation();
26  }
```

The nowait clause

- **What it does**
  - Without nowait: single has an **implicit barrier** at the end → all threads wait.
  - With nowait: **no barrier** → other threads continue immediately.
- Use nowait only if it's safe for threads to continue without waiting (no dependency on what the single thread does).



Who is in Charge?

- **Distinction:**
  - **#pragma omp** :
    - **Who:** Strictly **Thread 0**.
    - **Barrier: None.** Other threads do *not* wait; they keep running past the block immediately.
  - **#pragma omp single**:
    - **Who:** The **first thread** that arrives (could be Thread 0, Thread 5, etc.).
    - **Barrier: Yes.** Implicit barrier at the end (everyone waits).
  - **Guideline:**
    - Use when you want only the thread to run a block, and you don't want an implicit barrier that makes other threads wait.
    - Use single if you just need the job done safely by anyone and want others to wait for the result.

Runnable Example Code

single_vs_master.c

```c
int main() {
    // Set 4 threads for a clear demonstration
    omp_set_num_threads(4);

    printf("--- STARTING PARALLEL REGION ---\n");

    #pragma omp parallel
    {
        int id = omp_get_thread_num();

        // ------------------------------------------------------------
        // DEMO 1: #pragma omp single
        // "One person does the job, everyone else waits for them to finish."
        // ------------------------------------------------------------
        #pragma omp single
        {
            printf("[SINGLE] Thread %d is initializing shared data... (Taking 2 seconds)\n", id);
            sleep(2); // Simulate heavy initialization work
            printf("[SINGLE] Thread %d is DONE.\n", id);
        }
        // <--- IMPLICIT BARRIER HERE.
        // Threads cannot cross this line until the SINGLE block is finished.

        printf("Thread %d passed the 'single' barrier.\n", id);

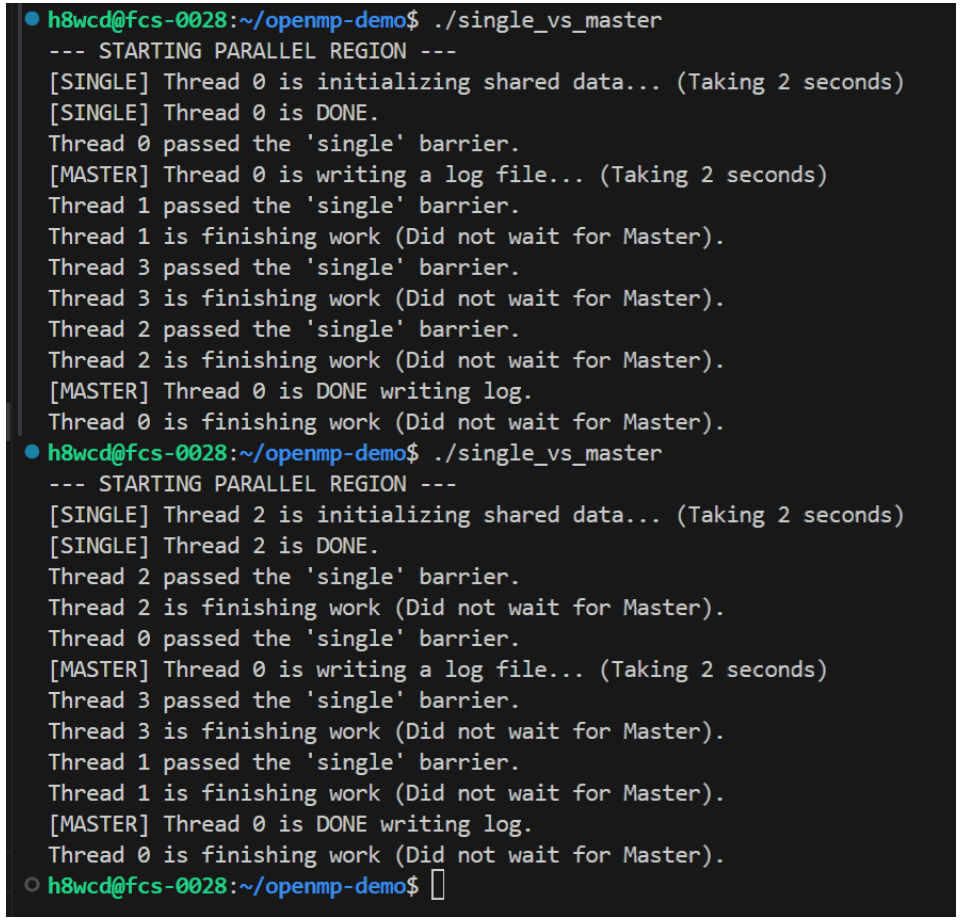

        // ------------------------------------------------------------
        // DEMO 2: #pragma omp master
        // "The Boss (Thread 0) does a job. Everyone else ignores it and keeps running."
        // ------------------------------------------------------------
        #pragma omp master
        {
            printf("[MASTER] Thread %d is writing a log file... (Taking 2 seconds)\n", id);
            sleep(2); // Simulate work
            printf("[MASTER] Thread %d is DONE writing log.\n", id);
        }
        // <--- NO BARRIER HERE.
        // Thread 1, 2, and 3 will NOT wait. They will print the line below immediately.

        printf("Thread %d is finishing work (Did not wait for Master).\n", id);
    }
```

```
h8wcd@fcs-0028:~/openmp-demo$ gcc -fopenmp single_vs_master.c -o single_vs_master
h8wcd@fcs-0028:~/openmp-demo$ ./single_vs_master
--- STARTING PARALLEL REGION ---
[SINGLE] Thread 0 is initializing shared data... (Taking 2 seconds)
[SINGLE] Thread 0 is DONE.
Thread 0 passed the 'single' barrier.
[MASTER] Thread 0 is writing a log file... (Taking 2 seconds)
Thread 1 passed the 'single' barrier.
Thread 1 is finishing work (Did not wait for Master).
Thread 3 passed the 'single' barrier.
Thread 3 is finishing work (Did not wait for Master).
Thread 2 passed the 'single' barrier.
Thread 2 is finishing work (Did not wait for Master).
[MASTER] Thread 0 is DONE writing log.
Thread 0 is finishing work (Did not wait for Master).
h8wcd@fcs-0028:~/openmp-demo$ ./single_vs_master
--- STARTING PARALLEL REGION ---
[SINGLE] Thread 2 is initializing shared data... (Taking 2 seconds)
[SINGLE] Thread 2 is DONE.
Thread 2 passed the 'single' barrier.
Thread 2 is finishing work (Did not wait for Master).
Thread 0 passed the 'single' barrier.
[MASTER] Thread 0 is writing a log file... (Taking 2 seconds)
Thread 3 passed the 'single' barrier.
Thread 3 is finishing work (Did not wait for Master).
Thread 1 passed the 'single' barrier.
Thread 1 is finishing work (Did not wait for Master).
[MASTER] Thread 0 is DONE writing log.
Thread 0 is finishing work (Did not wait for Master).
h8wcd@fcs-0028:~/openmp-demo$
```

The "Race" to the Console

- **The Concept:** When a barrier is released, all threads are unpaused simultaneously.
- **The Reality:** They do not wake up at the exact same nanosecond. The Operating System (OS) scheduler decides who gets the CPU first.
- **In the upper part of the screenshot:**
  1. **Thread 0** was the "Single" worker. It was already active (awake).
  2. **Threads 1, 2, 3** were waiting (asleep) at the barrier.
  3. **The Result:** Thread 0 was faster. It finished the single block, released the barrier, and immediately sprinted to the block to print its message *before* the other threads even had a chance to wake up and tell you they passed the barrier.



How to Force "Clean" Output (For Teaching)

```
28  #pragma omp parallel
29  {
30      // ... single block happens here ...
31
32      // 1. Force everyone to report "passed" before anyone moves on
33      printf("Thread %d passed the 'single' barrier.\n", omp_get_thread_num());
34
35      #pragma omp barrier  // <--- NEW BARRIER
36
37      // 2. Now the Master can start safely without racing ahead
38      #pragma omp master
39      {
40          printf("[MASTER] Thread %d is writing...\n", omp_get_thread_num());
41          sleep(2);
42      }
43  }
```

Back to the dependency example: The OpenMP 4.5 Solution (Manual Prefix Sum)

- This method breaks the dependency into three distinct phases:
  - **Phase 1 (Parallel):** Each thread calculates the prefix sum for its own *local* chunk. It stores the total sum of its chunk in a shared offsets array.
  - **Phase 2 (Serial):** One thread (usually the) calculates the prefix sum of those chunk totals to find the global starting point for each thread.
  - **Phase 3 (Parallel):** Each thread adds its global starting offset to every element in its chunk.

This needs to introduce new concepts

Back to the dependency example

```
18      #pragma omp parallel
19      {
20          int id = omp_get_thread_num();
21          int nthr = omp_get_num_threads();
22
23          // Calculate manual chunk boundaries
24          int chunk_size = N / nthr;
25          int start = id * chunk_size;
26          int end = (id == nthr - 1) ? N : start + chunk_size;
27
28          // --- PHASE 1: Local Prefix Sum ---
29          // Calculate the sum just for my slice of the array
30          // Treat c[start] as the first element (no dependency on previous chunk yet)
31          if (start < end) {
32              c[start] = a[start];
33              for (int i = start + 1; i < end; i++) {
34                  c[i] = c[i-1] + a[i];
35              }
36              // Store the total of this chunk in the offsets array
37              offsets[id] = c[end - 1];
38          } else {
39              offsets[id] = 0; // Handle empty chunks safety
40          }
41
42          // Wait for all threads to finish Phase 1
43          #pragma omp barrier
```

```
45          // --- PHASE 2: Calculate Global Offsets (Serial) ---
46          // Only one thread needs to do this fast calculation
47          #pragma omp single
48          {
49              // Convert the 'offsets' array into a prefix sum of itself
50              // offsets[0] stays as is (Thread 0 needs no offset)
51              // offsets[1] = offsets[0]
52              // offsets[2] = offsets[0] + offsets[1]
53              // etc...
54
55              // We need a temporary array or careful in-place logic.
56              // For simplicity, we'll do an in-place run:
57              int total_so_far = 0;
58              for (int i = 0; i < nthr; i++) {
59                  int temp = offsets[i];
60                  offsets[i] = total_so_far;
61                  total_so_far += temp;
62              }
63          }
64          // Implicit barrier at end of 'single' ensures offsets are ready
65
66          // --- PHASE 3: Apply Offsets ---
67          // Add the global offset to my local chunk (except Thread 0 which starts at 0)
68          if (id > 0) {
69              int my_offset = offsets[id];
70              for (int i = start; i < end; i++) {
71                  c[i] += my_offset;
72              }
73          }
74      }
```

```
h8wcd@fcs-0028:~/openmp-demo$ ./dependency2
Last element c[19] = 20 (Expected 20)
c[0]=1, c[4]=5, c[5]=6
h8wcd@fcs-0028:~/openmp-demo$
```

Solving Dependencies in OpenMP 5.0+ (The Easy Way)

- **The Old Way (< v5.0):** Required manual "Split-Scan-Merge" logic (3 distinct phases, ~50 lines of code).
- **The New Way (v5.0+):** Introduced the scan directive to handle this pattern automatically.
- **Key Syntax: reduction(inscan)**
- **Header:** Tell OpenMP this loop involves a scan operation.
  - #pragma omp parallel for reduction(inscan, +:sum)
- **Body:** distinct separation of "local work" and "global update."
  - #pragma omp scan inclusive(sum) acts as the synchronization point.

Ex. 1

- What values can x take?
- What values can shared take inside the printed lines?
- What values can Final shared be?
- Which parts are deterministic and which are non-deterministic? Explain briefly.

tutorial1b.c

```c
#include <stdio.h>
#include <omp.h>

int main() {

    omp_set_num_threads(4);

    int shared = 100;

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int x = 5;

        x = x + id;
        shared = shared + id;

        printf("id=%d x=%d shared=%d\n", id, x, shared);
    }

    printf("Final shared=%d\n", shared);
    return 0;
}

```

Ex. 1 key

- 1) Values of x
  - x is declared inside the parallel region → private stack variable (each thread has its own copy).
  - So each thread computes:
    - Thread 0: x = 5 + 0 = 5
    - Thread 1: x = 5 + 1 = 6
    - Thread 2: x = 5 + 2 = 7
    - Thread 3: x = 5 + 3 = 8
  - Possible printed x values: {5, 6, 7, 8}
  - Order is non-deterministic.

- **2) Values of shared inside the printed lines**
  - shared is declared **outside** the parallel region → **shared variable**.
  - All threads execute:
    - shared = shared + id;
  - This is a **race condition** (multiple threads write the same variable).
  - So in the print line:
    - printf("... shared=%d\n", shared);
  - shared could be many values depending on timing.
    - It will typically be in a range like:
    - minimum possible value: **100** (if a thread prints before any updates "appear")
    - maximum possible value: **106** (100 + (0+1+2+3) = 106) if all updates happened "nicely"
  - Possible shared values in prints: **could be 100 up to 106** (non-deterministic)

tutorial1b.c

```c
#include <stdio.h>
#include <omp.h>

int main() {

    omp_set_num_threads(4);

    int shared = 100;

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int x = 5;

        x = x + id;
        shared = shared + id;

        printf("id=%d x=%d shared=%d\n", id, x, shared);
    }

    printf("Final shared=%d\n", shared);
    return 0;
}

```

Ex1. key cont

- **3) Value of Final shared**
  - After the parallel region, the final value is also affected by the race.
  - So:
    - Final shared is **non-deterministic**, but usually ends up **somewhere between 100 and 106**.

- 4) Deterministic vs Non-deterministic
  - Deterministic
    - id is always one of {0,1,2,3}
    - x values are always {5,6,7,8} (one per thread)
  - Non-deterministic
    - the order of lines printed
    - the value of shared in each line (race condition)
    - Final shared (race condition)

Ex2

- You are given an array a[0..N-1]. Compute the **sum of all elements.**

ex2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int N = 10000000;
    double *a = malloc(N * sizeof(double));

    for (int i = 0; i < N; i++)
        a[i] = 1.0;

    double sum = 0.0;

    omp_set_num_threads(4);

    // TODO: parallelize this loop using OpenMP
    // Try your best "obvious" approach first.
    // (Do not use reduction yet.)
    for (int i = 0; i < N; i++) {
        sum += a[i];
    }

    printf("sum = %.0f (expected %.0f)\n", sum, (double)N);

    free(a);
    return 0;
}
```